OPERATING SYSTEMS

Incourse lab + viva | Five-topic exam pack

Exam-paper format, corrected C programs and worked results.

Scope: FCFS, nonpreemptive SJF, Round Robin, MFT and MVT only. Scheduling assumes all arrival times are zero, one CPU burst per process and no I/O or context-switch cost. Memory values use one consistent unit (for example KB) and represent memory available to processes after any OS reservation.

How to write your answer

Use these six headings in order: 1. Project name 2. Description 3. Algorithm 4. Source code 5. Input 6. Output. Add a one-line result if your examiner requests it.

Contents

1. FCFS CPU Scheduling: exam answer, full code, sample run and viva

2. Nonpreemptive SJF CPU Scheduling: exam answer, full code, sample run and viva

3. Round Robin CPU Scheduling: exam answer, full code, sample run and viva

4. MFT Memory Management: exam answer, full code, sample run and viva

5. MVT Memory Management: exam answer, full code, sample run and viva

Then: common viva, quick revision, compilation and publishing instructions.

Correct assumptions to state

TAT = CT - AT; WT = TAT - BT; RT = first start - AT. Here AT = 0, so TAT = CT. Do not use first start as RR waiting time. MFT internal waste excludes empty partitions and tail memory. MVT remaining free memory is not automatically external fragmentation. These are teaching simulations, not actual OS resource managers.

01 / FCFS CPU Scheduling

1. Project name

Implementation of FCFS CPU Scheduling using C

2. Description

Simulate First Come First Served scheduling and calculate waiting and turnaround times. All processes arrive at time zero, so input order is the service order. Each process runs to completion.

3. Algorithm

1. Start; read n and n positive burst times.

2. Set time = 0 and totals = 0.

3. For each process in input order, set WT = time.

4. Add BT to time; set CT = time and TAT = CT.

5. Add WT and TAT to their totals.

6. Print the process table and averages; stop.

Key variables

bt: burst; order: execution indices; time: simulated clock; ct: completion; wt: waiting; first: first start / response time.

4. Source code - fcfs.c

Copy the complete program, including headers and read_int(). Save each program separately. Prompts are sent to stderr; calculated output is sent to stdout.

```
/* Standalone C11 teaching simulator. Input formats: README.md. */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

/* Read bounded integer tokens; reject malformed or out-of-range input. */
static int read_int(const char *label, int low, int high) {
    char token[64], *end;
    long value;
    fprintf(stderr, "%s [%d..%d]: ", label, low, high);
    if (scanf("%63s", token) != 1) {
        fprintf(stderr, "Missing input.\n"); exit(EXIT_FAILURE);
    }
    errno = 0;
    value = strtol(token, &end, 10);
    if (errno || *end || value < low || value > high) {
        fprintf(stderr, "Invalid input: %s\n", token);
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

/* All arrivals are zero; no I/O or context-switch cost. */
int main(void) {
    int n = read_int("Process count", 1, 50);
    int bt[50], order[50], ct[50], wt[50], first[50];
    int time = 0;
    double sum_wt = 0, sum_tat = 0;
    for (int i = 0; i < n; ++i) {
        bt[i] = read_int("Burst", 1, 1000);
        order[i] = i;
    }
    for (int k = 0; k < n; ++k) {
        int i = order[k];
        first[i] = wt[i] = time;
        printf("RUN P%d %d %d\n", i+1, time, time+bt[i]);
        time += bt[i]; ct[i] = time;
    }
    puts("PID BT CT TAT WT RT");
    for (int i = 0; i < n; ++i) {
        printf("P%d %d %d %d %d %d\n", i+1, bt[i], ct[i],
               ct[i], wt[i], first[i]);
        sum_wt += wt[i]; sum_tat += ct[i];
    }
    printf("AVG_WT %.6f AVG_TAT %.6f\n", sum_wt/n, sum_tat/n);
    return 0;
}
```

5. Input

n, then n burst times.

```
3
5 3 2
```

6. Output

Exact calculated stdout for the sample input; interactive prompts are omitted. RUN lines show process, start and end. RT means response time.

```
RUN P1 0 5
RUN P2 5 8
RUN P3 8 10
PID BT CT TAT WT RT
P1 5 5 5 0 0
P2 3 8 8 5 5
P3 2 10 10 8 8
AVG_WT 4.333333 AVG_TAT 7.666667
```

Worked explanation / result

P1 runs 0-5, P2 runs 5-8, P3 runs 8-10. WT = 0, 5, 8. Average WT = 13/3 = 4.33. Average TAT = 23/3 = 7.67.

Viva preparation

1. Is FCFS preemptive?

No. It does not interrupt a burst to serve another ready process.

2. Which queue rule is used?

FIFO: First In, First Out.

3. What is the convoy effect?

Short processes wait behind a long process.

4. Why is the first WT zero?

The CPU is available and all arrivals are zero.

5. Does FCFS cause starvation?

Not for a queued process when jobs ahead finish in finite time.

6. What if arrivals differ?

Use start = max(previous CT, AT), CT = start + BT and WT = start - AT; order by arrival.

7. What is its complexity here?

O(n) time and O(n) array storage.

8. Why is input order used?

All arrivals tie at zero; input order is the chosen tie rule.

02 / Nonpreemptive SJF CPU Scheduling

1. Project name

Implementation of Nonpreemptive SJF CPU Scheduling using C

2. Description

Simulate Shortest Job First scheduling. All processes arrive at time zero. Select shorter bursts first; equal bursts retain input order. Each selected process runs to completion.

3. Algorithm

1. Start; read n and burst times; store each original process ID.

2. Create an order array containing the process indices.

3. Stable-sort the order array by increasing burst time.

4. Set time = 0. Visit processes in sorted order.

5. Set WT = time, add BT to time and set CT = TAT = time.

6. Print the table with original IDs and the averages; stop.

Key variables

order holds process indices. In insertion sort, x is the next index to insert and j is its position. IDs and bursts remain associated.

4. Source code - sjf.c

Copy the complete program, including headers and read_int(). Save each program separately. Prompts are sent to stderr; calculated output is sent to stdout.

```
/* Standalone C11 teaching simulator. Input formats: README.md. */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

/* Read bounded integer tokens; reject malformed or out-of-range input. */
static int read_int(const char *label, int low, int high) {
    char token[64], *end;
    long value;
    fprintf(stderr, "%s [%d..%d]: ", label, low, high);
    if (scanf("%63s", token) != 1) {
        fprintf(stderr, "Missing input.\n"); exit(EXIT_FAILURE);
    }
    errno = 0;
    value = strtol(token, &end, 10);
    if (errno || *end || value < low || value > high) {
        fprintf(stderr, "Invalid input: %s\n", token);
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

/* All arrivals are zero; no I/O or context-switch cost. */
int main(void) {
    int n = read_int("Process count", 1, 50);
    int bt[50], order[50], ct[50], wt[50], first[50];
    int time = 0;
    double sum_wt = 0, sum_tat = 0;
    for (int i = 0; i < n; ++i) {
        bt[i] = read_int("Burst", 1, 1000);
        order[i] = i;
    }
    /* Stable ascending-key insertion sort; IDs stay attached to data. */
    for (int i = 1; i < n; ++i) {
        int x = order[i], j = i;
        while (j > 0 && bt[order[j-1]] > bt[x]) {
            order[j] = order[j-1]; --j;
        }
        order[j] = x;
    }
    for (int k = 0; k < n; ++k) {
        int i = order[k];
        first[i] = wt[i] = time;
        printf("RUN P%d %d %d\n", i+1, time, time+bt[i]);
        time += bt[i]; ct[i] = time;
    }
    puts("PID BT CT TAT WT RT");
    for (int i = 0; i < n; ++i) {
        printf("P%d %d %d %d %d %d\n", i+1, bt[i], ct[i],
               ct[i], wt[i], first[i]);
        sum_wt += wt[i]; sum_tat += ct[i];
    }
    printf("AVG_WT %.6f AVG_TAT %.6f\n", sum_wt/n, sum_tat/n);
    return 0;
}
```

5. Input

n, then n burst times.

```
3
5 3 2
```

6. Output

Exact calculated stdout for the sample input; interactive prompts are omitted. RUN lines show process, start and end. RT means response time.

```
RUN P3 0 2
RUN P2 2 5
RUN P1 5 10
PID BT CT TAT WT RT
P1 5 10 10 5 5
P2 3 5 5 2 2
P3 2 2 2 0 0
AVG_WT 2.333333 AVG_TAT 5.666667
```

Worked explanation / result

Order: P3 (0-2), P2 (2-5), P1 (5-10). WT in original ID order = 5, 2, 0. Average WT = 7/3 = 2.33; average TAT = 17/3 = 5.67.

Viva preparation

1. Why choose the shortest burst?

With all jobs ready and bursts known, this order minimizes average waiting time.

2. What is the preemptive counterpart?

Shortest Remaining Time First, or SRTF.

3. Can SJF cause starvation?

Yes, in an ongoing workload if short jobs continually arrive ahead of a long job. The finite batch here finishes.

4. Why sort indices instead of changing IDs?

The order array selects execution order while burst times remain attached to original process IDs.

5. What if bursts tie?

Stable insertion sort preserves input order by shifting only strictly larger bursts.

6. Are bursts known in real systems?

Usually they must be estimated.

7. Does one global sort handle differing arrivals?

No. At each decision, choose only among processes already ready.

8. What is the sort complexity?

Worst-case O(n squared) time; scheduling after sorting is O(n).

03 / Round Robin CPU Scheduling

1. Project name

Implementation of Round Robin CPU Scheduling using C

2. Description

Simulate preemptive Round Robin scheduling. All processes arrive at time zero. Each unfinished process receives up to one time quantum per turn, in cyclic input order.

3. Algorithm

1. Start; read n, positive burst times and a positive quantum.

2. Copy bursts to remaining[]; set time = done = 0.

3. Scan processes cyclically, skipping completed processes.

4. Set run = minimum(remaining time, quantum).

5. Increase time by run and decrease remaining time by run.

6. When remaining becomes zero, record CT and increase done.

7. Repeat until done = n; calculate TAT = CT and WT = CT - BT.

8. Display results and averages; stop.

Key variables

remaining: unfinished burst; done: completed count; run: current slice; first initialized to -1 records the first start only once.

4. Source code - round_robin.c

Copy the complete program, including headers and read_int(). Save each program separately. Prompts are sent to stderr; calculated output is sent to stdout.

```
/* Standalone C11 teaching simulator. Input formats: README.md. */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

/* Read bounded integer tokens; reject malformed or out-of-range input. */
static int read_int(const char *label, int low, int high) {
    char token[64], *end;
    long value;
    fprintf(stderr, "%s [%d..%d]: ", label, low, high);
    if (scanf("%63s", token) != 1) {
        fprintf(stderr, "Missing input.\n"); exit(EXIT_FAILURE);
    }
    errno = 0;
    value = strtol(token, &end, 10);
    if (errno || *end || value < low || value > high) {
        fprintf(stderr, "Invalid input: %s\n", token);
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

/* All arrivals zero; initial ready order P1..Pn; zero switching cost. */
int main(void) {
    int n = read_int("Process count", 1, 50);
    int bt[50], remaining[50], ct[50], first[50];
    int time = 0, done = 0;
    double sw = 0, st = 0;
    for (int i = 0; i < n; ++i) {
        bt[i] = remaining[i] = read_int("Burst", 1, 1000);
        first[i] = -1;
    }
    int quantum = read_int("Quantum", 1, 1000);
    while (done < n) {
        for (int i = 0; i < n; ++i) {
            if (!remaining[i]) continue;
            if (first[i] == -1) first[i] = time;
            int run = remaining[i] < quantum ? remaining[i] : quantum;
            printf("RUN P%d %d %d\n", i+1, time, time+run);
            time += run; remaining[i] -= run;
            if (!remaining[i]) { ct[i] = time; ++done; }
        }
    }
    puts("PID BT CT TAT WT RT");
    for (int i = 0; i < n; ++i) {
        int wt = ct[i] - bt[i];
        printf("P%d %d %d %d %d %d\n", i+1, bt[i], ct[i], ct[i], wt, first[i]);
        sw += wt; st += ct[i];
    }
    printf("AVG_WT %.6f AVG_TAT %.6f\n", sw/n, st/n);
    return 0;
}
```

5. Input

n, then n burst times, then quantum.

```
3
5 3 2
2
```

6. Output

Exact calculated stdout for the sample input; interactive prompts are omitted. RUN lines show process, start and end. RT means response time.

```
RUN P1 0 2
RUN P2 2 4
RUN P3 4 6
RUN P1 6 8
RUN P2 8 9
RUN P1 9 10
PID BT CT TAT WT RT
P1 5 10 10 5 0
P2 3 9 9 6 2
P3 2 6 6 4 4
AVG_WT 5.000000 AVG_TAT 8.333333
```

Worked explanation / result

With quantum 2: P1 0-2; P2 2-4; P3 4-6; P1 6-8; P2 8-9; P1 9-10. CT = 10, 9, 6; WT = 5, 6, 4. Average WT = 5.00; average TAT = 8.33.

Viva preparation

1. What is a quantum?

The maximum CPU time granted in one turn.

2. What does preemption mean?

The OS interrupts a running process, saves its state and lets another run. The interrupted process resumes later.

3. Why keep original burst times?

Remaining times decrease; original bursts are required for WT = CT - BT.

4. What if the quantum is very large?

If it is at least every burst, this model behaves like FCFS.

5. What if the quantum is very small?

Response can be quicker, but real context-switch overhead increases.

6. Why reject zero quantum?

Execution would make no progress and the loop would not terminate.

7. Is response time equal to waiting time?

Not generally. Response is time before first execution; WT includes waiting between turns.

8. Can a turn be shorter than the quantum?

Yes, when the remaining burst is smaller.

9. Why use a repeated scan?

All processes arrive at zero and unfinished jobs remain in fixed cyclic order.

10. What does done count?

Processes whose remaining burst has reached zero.

04 / MFT Memory Management

1. Project name

Implementation of MFT Memory Management using C

2. Description

Simulate Multiprogramming with a Fixed Number of Tasks using equal fixed partitions. Each admitted process occupies one partition. Report internal fragmentation, unused partitions and the unpartitioned tail separately.

3. Algorithm

1. Start; read total available memory, partition size and process count.

2. Compute count = total / block and tail = total % block.

3. Set used partitions and internal waste to zero.

4. Read each process size. If size > block, report TOO_LARGE.

5. Otherwise, if used = count, report NO_PARTITION.

6. Otherwise allocate one partition, increase used and add block - size to waste.

7. Print internal waste, count - used unused partitions and tail; stop.

Key variables

count: whole partitions; block: partition size; tail: unpartitioned remainder; used: occupied partitions; waste: internal fragmentation.

4. Source code - mft.c

Copy the complete program, including headers and read_int(). Save each program separately. Prompts are sent to stderr; calculated output is sent to stdout.

```
/* Standalone C11 teaching simulator. Input formats: README.md. */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

/* Read bounded integer tokens; reject malformed or out-of-range input. */
static int read_int(const char *label, int low, int high) {
    char token[64], *end;
    long value;
    fprintf(stderr, "%s [%d..%d]: ", label, low, high);
    if (scanf("%63s", token) != 1) {
        fprintf(stderr, "Missing input.\n"); exit(EXIT_FAILURE);
    }
    errno = 0;
    value = strtol(token, &end, 10);
    if (errno || *end || value < low || value > high) {
        fprintf(stderr, "Invalid input: %s\n", token);
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

/* Equal fixed partitions; no freeing; unused tail is reported separately. */
int main(void) {
    int total = read_int("Total memory", 1, 1000000);
    int block = read_int("Partition size", 1, total);
    int n = read_int("Process count", 1, 100);
    int count = total / block, tail = total % block, used = 0, waste = 0;
    printf("PARTITIONS %d TAIL %d\n", count, tail);
    puts("PID REQUEST RESULT INTERNAL_WASTE");
    for (int i = 0; i < n; ++i) {
        int size = read_int("Request", 1, 1000000);
        if (size > block) printf("P%d %d TOO_LARGE -\n", i+1, size);
        else if (used == count) printf("P%d %d NO_PARTITION -\n", i+1, size);
        else {
            ++used; waste += block-size;
            printf("P%d %d ALLOCATED %d\n", i+1, size, block-size);
        }
    }
    printf("INTERNAL_WASTE %d UNUSED_PARTITIONS %d TAIL %d\n", waste, count-used, tail);
    return 0;
}
```

5. Input

Total memory, partition size, n, then n process sizes.

```
1000
300
5
275 400 290 293 100
```

6. Output

Exact calculated stdout for the sample input; interactive prompts are omitted. RUN lines show process, start and end. RT means response time.

```
PARTITIONS 3 TAIL 100
PID REQUEST RESULT INTERNAL_WASTE
P1 275 ALLOCATED 25
P2 400 TOO_LARGE -
P3 290 ALLOCATED 10
P4 293 ALLOCATED 7
P5 100 NO_PARTITION -
INTERNAL_WASTE 42 UNUSED_PARTITIONS 0 TAIL 100
```

Worked explanation / result

1000/300 gives 3 whole partitions and a tail of 100. P1, P3 and P4 occupy those partitions. Internal waste = 25 + 10 + 7 = 42. P2 is too large; P5 arrives after all partitions are used.

Viva preparation

1. What is internal fragmentation?

Unused space inside an allocated partition: block size minus process size.

2. Must fixed partitions be equal?

No. MFT may use unequal fixed partitions; this program uses equal ones.

3. Why can allocation fail with free memory?

A process must fit one partition; it cannot combine several partitions in this model.

4. What limits resident processes?

The number of partitions, with one process per partition.

5. Is an unused partition internal waste?

No. It is unallocated memory, not slack inside an allocation.

6. What does % calculate?

The remainder after forming whole fixed partitions.

7. Is tail memory internal fragmentation?

No. It is outside the formed partitions and is reported separately.

8. Does the program release partitions?

No. It models admission only; a full simulator must track termination and reuse.

05 / MVT Memory Management

1. Project name

Implementation of MVT Memory Management using C

2. Description

Simulate Multiprogramming with a Variable Number of Tasks using a simple single free-memory pool. Allocate exactly the requested amount when it fits. This accounting model does not simulate holes, deallocation or compaction.

3. Algorithm

1. Start; read total available memory and process count.

2. Set remaining = total.

3. For each request, compare size with remaining.

4. If size <= remaining, allocate and subtract size from remaining.

5. Otherwise report NOT_ALLOCATED; continue to later requests.

6. Print allocated memory = total - remaining and remaining free memory; stop.

Key variables

total: initial available memory; remaining: free pool; size: current request. total - remaining is allocated memory.

4. Source code - mvt.c

Copy the complete program, including headers and read_int(). Save each program separately. Prompts are sent to stderr; calculated output is sent to stdout.

```
/* Standalone C11 teaching simulator. Input formats: README.md. */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

/* Read bounded integer tokens; reject malformed or out-of-range input. */
static int read_int(const char *label, int low, int high) {
    char token[64], *end;
    long value;
    fprintf(stderr, "%s [%d..%d]: ", label, low, high);
    if (scanf("%63s", token) != 1) {
        fprintf(stderr, "Missing input.\n"); exit(EXIT_FAILURE);
    }
    errno = 0;
    value = strtol(token, &end, 10);
    if (errno || *end || value < low || value > high) {
        fprintf(stderr, "Invalid input: %s\n", token);
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

/* Simplified variable-size allocation from ONE free pool, no deallocation.
   This is a single-pool model, not a fragmented hole list.
   A failed request does not stop later requests. */
int main(void) {
    int total = read_int("Total memory", 1, 1000000), remaining = total;
    int n = read_int("Process count", 1, 100);
    puts("PID REQUEST RESULT FREE_REMAINING");
    for (int i = 0; i < n; ++i) {
        int size = read_int("Request", 1, 1000000);
        if (size <= remaining) {
            remaining -= size;
            printf("P%d %d ALLOCATED %d\n", i+1, size, remaining);
        } else printf("P%d %d NOT_ALLOCATED %d\n", i+1, size, remaining);
    }
    printf("ALLOCATED %d FREE %d\n", total-remaining, remaining);
    return 0;
}
```

5. Input

Total memory, n, then n process sizes.

```
1000
4
400 275 550 200
```

6. Output

Exact calculated stdout for the sample input; interactive prompts are omitted. RUN lines show process, start and end. RT means response time.

```
PID REQUEST RESULT FREE_REMAINING
P1 400 ALLOCATED 600
P2 275 ALLOCATED 325
P3 550 NOT_ALLOCATED 325
P4 200 ALLOCATED 125
ALLOCATED 875 FREE 125
```

Worked explanation / result

After P1 and P2, 325 units remain. P3 requests 550 and fails. P4 requests 200 and succeeds. Allocated memory = 875; free memory = 125. A failed request does not consume memory.

Viva preparation

1. How does MVT differ from MFT?

MVT sizes allocations to requests; MFT uses predetermined partitions.

2. Does ideal exact-size MVT have internal waste?

No partition-size waste; real systems can have alignment and metadata overhead.

3. What is external fragmentation?

Free space is split into holes so no single hole satisfies a contiguous request even if the total free space is enough.

4. Is all remaining memory external fragmentation?

No. A single free region is not evidence of fragmentation.

5. Does this program demonstrate scattered holes?

No. A single remaining counter cannot describe hole locations or sizes.

6. What is compaction?

Relocating allocated regions together to combine separated free holes.

7. Why continue after a failed request?

A later smaller process may fit the remaining space.

8. What does a full MVT simulator need?

Allocated-region and free-hole lists, a placement policy, deallocation and adjacent-hole merging.

Common viva answers

1. What is an operating system?

System software that manages hardware resources and provides services to applications.

2. What is a process?

A program in execution, including its state and resources.

3. What is the ready queue?

Processes ready to execute but waiting for CPU time.

4. What is CPU scheduling?

Selecting a ready process to run on the CPU.

5. What is burst time?

CPU execution time needed by a process for a burst.

6. What is turnaround time?

Time from arrival until completion: TAT = CT - AT.

7. What is waiting time?

Total ready-queue waiting. In the single-burst model, WT = TAT - BT.

8. What is response time?

First CPU start time minus arrival time.

9. Why use double totals?

Dividing a double total by n retains a fractional average.

10. Do these programs change the real OS?

No. They simulate decisions and memory accounting.

11. What is multiprogramming?

Keeping multiple programs in memory so another can run when one waits.

12. What is a context switch?

Saving one execution context and restoring another; it takes time in real systems.

13. What does read_int do?

It reads one integer, validates its range and exits on invalid or missing input.

14. Why are array indices zero-based?

C arrays start at zero; printing index + 1 gives familiar process labels.

Final revision + practice

TAT = CT - AT; WT = TAT - BT; RT = first start - AT. Here AT = 0, so TAT = CT. Do not use first start as RR waiting time. MFT internal waste excludes empty partitions and tail memory. MVT remaining free memory is not automatically external fragmentation. These are teaching simulations, not actual OS resource managers.

FCFS CPU Scheduling

P1 runs 0-5, P2 runs 5-8, P3 runs 8-10. WT = 0, 5, 8. Average WT = 13/3 = 4.33. Average TAT = 23/3 = 7.67.

Nonpreemptive SJF CPU Scheduling

Order: P3 (0-2), P2 (2-5), P1 (5-10). WT in original ID order = 5, 2, 0. Average WT = 7/3 = 2.33; average TAT = 17/3 = 5.67.

Round Robin CPU Scheduling

With quantum 2: P1 0-2; P2 2-4; P3 4-6; P1 6-8; P2 8-9; P1 9-10. CT = 10, 9, 6; WT = 5, 6, 4. Average WT = 5.00; average TAT = 8.33.

MFT Memory Management

1000/300 gives 3 whole partitions and a tail of 100. P1, P3 and P4 occupy those partitions. Internal waste = 25 + 10 + 7 = 42. P2 is too large; P5 arrives after all partitions are used.

MVT Memory Management

After P1 and P2, 325 units remain. P3 requests 550 and fails. P4 requests 200 and succeeds. Allocated memory = 875; free memory = 125. A failed request does not consume memory.

Practice order

Write FCFS from memory first, then SJF sorting, RR remaining-time loop, MFT partition calculations and MVT subtraction. For each, explain every variable and reproduce its sample output without looking.

Compile and run

```
gcc -std=c11 -Wall -Wextra fcfs.c -o fcfs
./fcfs < fcfs.in
# Windows PowerShell: compile, then run .\fcfs.exe
# and type the sample inputs interactively.
```

Input validation

The shared read_int helper uses strtol to reject malformed and out-of-range tokens. errno reports conversion overflow; end points after the parsed number. Bursts and quantum are 1-1000; scheduling count is 1-50. Memory and requests are 1-1000000; memory process count is 1-100. The MFT partition must not exceed total memory.

Publishing

Extract the ZIP into your website repository. Preserve Semester-5/operating-system/. Link to os-incourse-exam.html from your dashboard. Keep PDF and code ZIP beside the HTML. The page works offline and has no external dependencies. This pack does not overwrite the earlier full OS guide.